

P3: Google Page Rank

Lily Nguyen and Andre Sae

November 2025

1 Instructions

In the third paper, you will explore how the Google PageRank algorithm works. A detailed description of this project is given in your book at the end of Chapter 4. This is a guided project, and you will follow the exercises given below:

- (Exercise 4.95) Finish writing the hyperlink matrix $\mathbf{H}$ from Figure 2.
- (Exercise 4.96) Write code to implement the iterative process defined previously. Then, create a plot showing how the rank evolves over the iterations.
- (Exercise 4.97) What must be true about a collection of n pages such that an $n \times n$ hyperlink matrix $\mathbf{H}$ is a stochastic matrix?
- (Exercise 4.98) The statement of Theorem 4.9 is incomplete, but the proof is given to you. Fill in the blank in the statement of the theorem, and provide a few sentences supporting your answer.
- (Exercise 4.99) Discuss how Theorem 4.9 greatly simplifies the PageRank iterative process described previously. In other words, there is no reason to iterate at all. Instead, find ... what?
- (Exercise 4.100) Use the previous two problems to find the resulting PageRank vector from the web in Figure 2. Be sure to rank the pages in order of importance.
- (Exercise 4.101) Consider the web in Figure 1.
 - Write the $\mathbf{H}$ matrix and find the initial state $\mathbf{x}_0$.
 - Find steady state PageRank vector using the two methods described: one using the iterative difference equation and the other using Theorem 4.9 and the dominant eigenvector.
 - Rank the pages in order of importance.

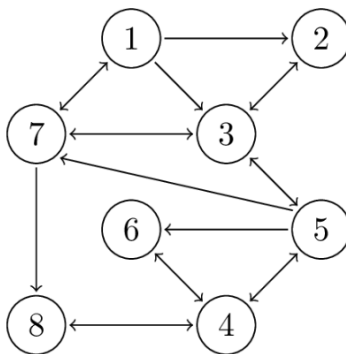


Figure 1: Example web graph.

Research other references to the Google PageRank algorithm. After you complete your investigations, you will write a research paper. The tone of your paper is critical to us. Browse papers in the Astrophysical Journal or Physical Review to get a feel for the writing style that we are expecting. You will be submitting a single paper for your group. We have enclosed the LaTeX template that you have to use. Here is the outline of your paper:

- Title
- Authors' Names
- Affiliation Name (The University of Texas at Austin)
- Abstract
- Keywords
- Introduction—In your introduction, you will research the background of this topic and summarize the importance of this algorithm in surfing the web. We want references to papers in published journals, not blogs on the Internet.
- You will state the algorithm and the code that you developed.
- You will present and discuss your results.
- Conclusion - You will conclude by reflecting on how you can improve (if at all) the page rank of your website. You will discuss any future work you could have done by adding randomness to the algorithm Ex 4.100 (book) or EX 4.102 (online).
- Acknowledgment - You will acknowledge any help from your classmates or the TAs.
- References

2 Problem

In this project you will discover how the Page Rank algorithm works to give the most relevant information as the top hit on a Google search.

Search engines compile large indexes of the dynamic information on the Internet so they are easily searched. This means that when you do a Google search, you are not actually searching the Internet; instead, you are searching the indexes at Google.

When you type a query into Google the following two steps take place:

1. **Query Module:** The query module at Google converts your natural language into a language that the search system can understand and consults the various indexes at Google in order to answer the query. This is done to find the list of relevant pages.
2. **Ranking Module:** The ranking module takes the set of relevant pages and ranks them. The outcome of the ranking is an ordered list of web pages such that the pages near the top of the list are most likely to be what you desire from your search. This ranking is the same as assigning a popularity score to each web site and then listing the relevant sites by this score.

This section focuses on the Linear Algebra behind the Ranking Module developed by the founders of Google: Sergey Brin and Larry Page. Their algorithm is called the Page Rank algorithm, and you use it every single time you use Google's (insert apostrophe) search engine.

In simple terms: A webpage is important if it is pointed to by other important pages.

The Internet can be viewed as a directed graph (look up this term here on Wikipedia) where the nodes are the web pages and the edges are the hyperlinks between the pages. The hyperlinks into a page are called in links, and the ones pointing out of a page are called out links. In essence, a hyperlink from my page to yours is my endorsement of your page. Thus, a page with more recommendations must be more important than a page with a few links. However, the status of the recommendation is also important.

Let us now translate this into mathematics. To help understand this we first consider the small web of six pages shown in Figure 2 (a graph of the router level of the internet can be found here). The links between the pages are shown by arrows. An arrow pointing into a node is an in link and an arrow pointing out of a node is an out link. In Figure 2, node 3 has three out links (to nodes 1, 2, and 5) and 1 in link (from node 1).

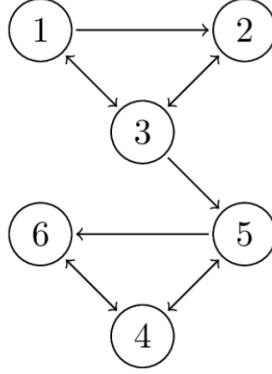


Figure 2: Example web graph.

We will first define some notation in the Page Rank algorithm:

- $|P_i|$ is the number of out links from page P_i
- $\mathbf{H}$ is the *hyperlink* matrix defined as:

$$\mathbf{H}_{ij} = \begin{cases} \frac{1}{|P_j|}, & \text{if there is a link from node } j \text{ to node } i \\ 0, & \text{otherwise} \end{cases}$$

where the i and j are the row and column indices respectively.

- $\mathbf{x}$ is a vector that contains all of the Page Ranks for the individual pages.

The Page Rank algorithm works as follows:

1. Initialize the page ranks to all be equal. This means that our initial assumption is that all pages are of equal rank. In the case of Figure 2 we would take $\mathbf{x}_0$ to be

$$\mathbf{x}_0 = \begin{pmatrix} 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \end{pmatrix}.$$

2. Build the hyperlink matrix. As an example, we'll consider node 3 in Figure 2. There are three out links from node 3 (to nodes 1, 2, and 5). Hence, $\mathbf{H}_{13} = 1/3$, $\mathbf{H}_{23} = 1/3$, and $\mathbf{H}_{53} = 1/3$ and the partially complete hy-

perlink matrix is

$$\mathbf{H} = \begin{pmatrix} - & - & 1/3 & - & - & - \\ - & - & 1/3 & - & - & - \\ - & - & 0 & - & - & - \\ - & - & 0 & - & - & - \\ - & - & 1/3 & - & - & - \\ - & - & 0 & - & - & - \end{pmatrix}$$

3. The difference equation $\mathbf{X}_{n+1} = \mathbf{H}\mathbf{x}_n$ is used to iteratively refine the estimates of the page ranks. You can view the iterations as a person visiting a page and then following a link at random, then following a random link on the next page, and the next, and the next, etc. Hence we see that the iterations evolve exactly as expected for a difference equation.

Iteration	New Page Rank Estimation
0	x_0
1	$x_1 = \mathbf{H}x_0$
2	$x_2 = \mathbf{H}x_1 = \mathbf{H}^2x_0$
3	$x_3 = \mathbf{H}x_2 = \mathbf{H}^3x_0$
4	$x_4 = \mathbf{H}x_3 = \mathbf{H}^4x_0$
$\vdots$	$\vdots$
k	$x_k = \mathbf{H}^kx_0$

4. When a steady state is reached we sort the resulting vector $\mathbf{x}_k$ to give the page rank. The node (web page) with the highest rank will be the top search result, the second highest rank will be the second search result, and so on.

It doesn't take much to see that this process can be very time consuming. Think about your typical web search with hundreds of thousands of hits; that makes a square matrix $\mathbf{H}$ that has a size of hundreds of thousands of entries by hundreds of thousands of entries! The matrix multiplications alone would take many minutes (or possibly many hours) for every search! ... but Brin and Page were pretty smart dudes!!

We now state a few theorems and definitions that will help us simplify the iterative Page Rank process.

3 Theorems

Theorem 4.7

If $\mathbf{A}$ is an $n \times n$ matrix with n linearly dependent eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \dots, \mathbf{v}_n$ and associated eigenvalues $\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_n$ then for any initial vector $\mathbf{x} \in \mathbb{R}^n$

we can write $\mathbf{A}^k \mathbf{x}$ as

$$\mathbf{A}^k \mathbf{x} = c_1 \lambda_1^k \mathbf{v}_1 + c_2 \lambda_2^k \mathbf{v}_2 + c_3 \lambda_3^k \mathbf{v}_3 + \cdots + c_n \lambda_n^k \mathbf{v}_n$$

where $c_1, c_2, c_3, \dots, c_n$ are the constants found by expressing $\mathbf{x}$ as a linear combination of the eigenvectors.

Note: We can assume that the eigenvalues are ordered such that $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$.

Theorem 4.8

A **probability vector** is a vector with entries on the interval $[0, 1]$ that add up to 1.

A **stochastic matrix** is a square matrix whose columns are probability vectors.

If $\mathbf{A}$ is a stochastic $n \times n$ matrix then $\mathbf{A}$ will have n linearly independent eigenvectors. Furthermore, the largest eigenvalue of a stochastic matrix will be $\lambda_1 = 1$ and the smallest eigenvalue will always be nonnegative: $0 \leq |\lambda_n| < 1$.

Theorem 4.9

If $\mathbf{A}$ is an $n \times n$ stochastic matrix and $\mathbf{x}_0$ is some initial vector for the difference equation $\mathbf{x}_{n+1} = \mathbf{A} \mathbf{x}_n$, then the steady state vector is

$$\mathbf{x}_{\text{equilibrium}} = \lim_{k \rightarrow \infty} \mathbf{A}^k \mathbf{x}_0 = ?$$

Proof:

First note that $\mathbf{A}$ is an $n \times n$ stochastic matrix so from Theorem 4.8. We know that there are n linearly independent eigenvectors. We can then substitute the eigenvalues from Theorem 4.8 in Theorem 4.7. Noting that if $0 < \lambda_j < 1$ we have $\lim_{k \rightarrow \infty} \lambda_j^k = 0$ the result follows immediately.

4 Extra

Exercise 4.102

One thing that we didn't consider in this version of the Google Page Rank algorithm is the random behavior of humans. One, admittedly slightly naive, modification that we can make to the present algorithm is to assume that the person surfing the web will randomly jump to any other page in the web at any time. For example, if someone is on page 1 in Figure 1 then they could randomly jump to any page 2 - 8. They also have links to pages 2, 3, and 7. That is a total of 10 possible next steps for the web surfer. There is a 2/10

chance of heading to page 2. One of those is following the link from page 1 to page 2 and the other is a random jump to page 2 without following the link. Similarly, there is a $2/10$ chance of heading to page 3, $2/10$ chance of heading to page 7, and a $1/10$ chance of randomly heading to any other page.

Implement this new algorithm, called the random surfer algorithm, on the web in Figure 1. Compare your ranking to the non-random surfer results from the previous problem.